



Magi：多智能體協作框架的架構設計與開放原始碼學習歷程

專案簡介

這個專案算是一個實驗性的作品，目的在讓我深度研究 Coding Agent 的架構設計與實作細節，同時也让我實驗對記憶管理和多agent的創新想法。這個項目的宗旨是打造一個名為「Magi」的多智能體協作框架。它串接 Azure OpenAI (o4-mini)，讓多個由 LLM 驅動的 Agent 能夠互相協作，解決複雜的寫程式與終端機任務。系統中的所有 Agent 皆繼承相同的核心理別 (agent)，共享工具集與記憶子系統，但在執行時擁有彼此獨立的上下文視窗 (Context Window)、短期記憶 (STM) 與長期記憶 (LTM) 權限。

創新架構一：Agentic 去中心化設計

如果只用單一個強大的 Agent 來處理所有事情，當任務變得複雜（如長期的終端機腳本執行、除錯、編輯程式碼等）時，很容易超出 Token 限制並導致邏輯混亂。因此我在這個專案實驗了一種去中心化的多智能體架構(靈感來源於我之前的專案llm世界引擎)。每個 Agent 都是獨立同等的實體，且可以互相分工或是溝通，也可以用內部工具動態創建新的 Agent 來協助處理特定任務。

在主程式 `main.py` 中，系統透過一個 Main Loop 不斷巡覽所有註冊代理人的狀態。當面對複雜工作時，收到使用者訊息的agent可以呼叫內部工具 `make_new_agent` 動態創建新的同等級智能體來分擔任務，或是透過 `send_message` 工具進行和現有的agent通訊，這個工具也負責傳訊息給user(因為專案採用reAct架構，沒辦法直接輸出訊息給user)。當傳送給其他 Agent 時，系統會以 `role: "agent name"` 的形式將訊息塞入目標 Agent 的歷史紀錄中。這種做法能讓 LLM 誤以為收到了來自使用者的直接指令，同時也能區分不同 Agent 的身份。接下來收到訊息的agent會變成啟動狀態，然後在下一個main loop啟動，如此循環。

創新架構二：模組化的記憶系統 (STM 與 LTM)

那陣子「Agent Skill」的概念在 AI 應用圈非常流行。所謂的 Agent Skill，是指將針對特定任務的工具、提示詞 (Prompt) 與運作規則封裝成獨立且模組化的「技能包」。當 Agent 遇到特定情境時，才動態將對應的技能載入到自身上下文中進行「裝備」，而不是一開始就在系統提

示詞 (System Prompt) 裡塞滿所有預設規則。這種機制能大幅減少冗餘的 Token 消耗，並有效避免模型因為無關資訊過多而導致注意力失焦或產生幻覺。

受到這種模組化設計的啟發，我就想嘗試看看：如果用類似掛載 Skill 的架構來動態管理長期記憶(ltm)，會有什麼樣的效果？就有了現在的 ltm 系統

Agent Context = LTM (長期記憶) + STM (短期記憶) + History (歷史訊息)

短期記憶 (STM) 存在於 Agent Process 內的輕量化記憶。Agent 可以利用 `remember(text)` 工具動態將發現的重點寫入自己的 `stm_content`。當對話超過設定的閥值時，系統會自動呼叫內部壓縮機制 (`compress_stm()`)，利用 LLM 將歷史訊息與 STM 重新摘要，確保對話永遠不會超載。同時，原始對話會被 Dump 成 JSON 存放於 `messages_log/` 讓我 debug。

長期記憶 (LTM)：這是這個專案最核心的設計。LTM 可以用來當system prompt設定agent或儲存系統規則、工具用法與跨任務的持久知識。這部分被設計為純文字的 Markdown 檔案 (位於 `ltm/`)，並以 Pydantic 的模型做資料結構化。我在實作時加入了兩個設計：

- **YAML Frontmatter 權限路由**：每個 `.md` 檔案頂端都包含 YAML 標籤。透過 `active_for` (強制自動載入到 System Prompt)、`visible_to` (Agent 可以看見名字跟描述，但不主動載入，需依情境呼叫 `active_ltm` 工具動態掛載) 與 `except_for` (黑名單隱藏)，系統的 `ltm_loader.py` 控制 Context 注入。這讓每個 Agent 不會被不相關的記憶污染 Token，並擁有各自專屬的運行規則與認知邊界。這個功能讓ltm可以取代傳統寫死的 system prompt，讓agent有辦法跟更彈性的管理自己的記憶跟規則。
- **基於 LTM 驅動的職責分離 (預設分工)**：透過上述的 LTM YAML 權限機制，這個架構可以透過載入不同的 ltm 檔案(system prompt)，用ltm系統完成職責分離。像 Magi-01 等一般智能體作為沙盒內的開發主力，它一般不會自己修改 ltm 檔。當遇到需要保留的系統級設定或記憶時，它會透過 Agent 之間的 `send_message` 工具發送訊息給**記憶管理員 (LTM-Manager)**。LTM-Manager 是一個預設的後台 Agent。系統透過 `active_for: [LTM-Manager]` 為它載入專屬的 ltm: `01_ltm_manager_rule.md` 當作 agent 設定。它不寫原本的 Code，而是專門用來接收其他 Agent 的委託並處理 `ltm/` 資料夾裡的記憶檔案。這解決了分散式架構中極易發生的 Race Condition (競態條件) 問題，也可以解決普通 Agent 不知道如何寫 LTM 檔的問題。

底層技術實作細節

1. 結構化行為控制與reAct互動模式

為了確保 Agent 動作的可靠性，我捨棄了傳統的純文本 Prompt 解析，而是定義了名為 `AgentStep` 的 Pydantic 模型（包含 `reasoning`、`tool_name`、`tool_args` 欄位）。每次呼叫 LLM 時，都會強制要求其輸出符合此格式。這等於在架構層面強制模型在呼叫工具前必須先在 `reasoning` 中進行自我盤點與反思，強制讓 ai 思考再使用工具。

2. PTY 終端機狀態管理机制

在處理終端機互動時，若僅用 `subprocess.run` 只能跑一次性指令。我參考了開源專案的做法，寫出了 `pty_manager.py`（PTY Manager）。這個模組利用 `pty.fork()` 捕捉完整的終端機輸出，並透過 `CommandSession` 物件管理每一個子程序。這讓 Agent 可以使用 `run_command` 啟動一個長時間運行的伺服器或腳本，將其放至背景，隨後透過 `command_status` 輪詢狀態，甚至透過 `send_command_input` 寫入 Standard Input (如回答互動式 CLI 的選項)。這讓 AI 操作終端機的模式與真實人類開發者如出一轍。

3. 內外部工具集

根據架構職責，系統的工具被拆分為兩大類：

第一大類是**內部系統與記憶工具**，直接實作於代理人核心（`agent` 類別）中，用於管理自身狀態。例如讓 Agent 主動將發現寫入短期記憶（`remember`）、主動壓縮記憶對話防超載（`compress_stm`）、結束當前任務轉入閒置狀態（`wait`）、以及在 Runtime 動態創建並啟動新代理人（`make_new_agent`）。

第二大類則是**外部操作與檔案工具**，被集中實作於 `tools.py` 中，並嚴格限制在專案沙盒環境內運行。外部工具除了前述的終端機執行與狀態輪詢，還提供了完整的檔案操作與檢索能力。這包含了精確的字串替換編輯（`edit_file`）、讀寫功能（`read_file`、`write_to_file`）、以及環境檢索（`ls`、`grep`），讓 Agent 修改檔案與尋找 Bug 的行為如同真實開發者般極具彈性。

心得與反思

開發 Magi 是我第一次嘗試將 Agentic 這樣的進階 AI 設計思維，真正落實到底層程式架構中。在實作過程中，我最大的成長之一是學會放下凡事都要自己重新造輪子的固執，開始懂得去拆解

並學習別人優秀的開源專案。

然而，如果以最初實驗的目標來看，這次的專案坦白說並不完全成功。我把ai的行動邏輯設計得太過龐大複雜，反而導致 ai 很容易失控或是忘記如何工作。這讓我領悟到，開發 ai 應用的關鍵其實是讓 **ai 的互動環境越單純越好**，這也是近期開發圈常提到的「Harness」概念，不然就是我得把提示詞寫到複雜且沒有疏漏，但這實在有點難。

當初投入這個專案，本來是希望能親手打造一個專屬的 Coding Agent，親自掌控提示詞設定、工具調用和長期記憶管理。但實際走過一遍才發現，底層細節遠比想像中深奧。Magi 完工後的實際效能，不僅與市面主流的工具仍有不小差距，我也沒有徹底解決長期記憶這個難題。

我打算接下來去嘗試文檔驅動開發(Document-Driven Development, DDD)的思維，試著讓 Agent 透過管理專案內特製的技術文檔（design docs），來作為維持專案脈絡的記憶載體。

整體而言，儘管 Magi 最終的效能表現不算完美，但它作為一個探索型專案，讓我從零開始剖析了現代 AI 代理系統的底層邏輯，並親手實作出了一套完整的 ReAct 互動模式，算是挺有收穫吧。

參考資料

開源coding agent(opencode):

我的github倉庫: